UNIVERSIDAD DE COSTA RICA
Sede del Atlántico

RP Recinto de Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 15: Sistema Móvil de Inspecciones Técnicas en Campo con Arquitectura Offline-First, GPS, Fotos y Backend Spring Boot 3 ("FieldTech Móvil")

Resumen

En este laboratorio guiado, usted construirá de principio a fin una solución móvil empresarial completa: **FieldTech Móvil**. El sistema está diseñado para que el personal técnico en campo registre reportes de incidencias físicas (averías en postes, cableado o antenas) incluso cuando carece totalmente de conexión a Internet.

La solución se compone de dos componentes independientes: un servidor **Back-End en Spring Boot 3** estructurado bajo una estricta arquitectura en capas (`domain`, `controller`, `business`, `data`)

con persistencia en **H2 Database**, y una aplicación **Front-End Móvil en Flutter 3 / Dart 3** que implementa el patrón **Offline-First** utilizando una base de datos local embebida en **SQLite (`sqflite`)**, captura de coordenadas satelitales en tiempo real con **GPS (`geolocator`)**, registro de evidencia fotográfica con **cámara (`image_picker`)**, y sincronización asíncrona en segundo plano con el servidor mediante peticiones multipart.

Metadatos del Laboratorio

- **Tiempo estimado de desarrollo:** 4.5 horas (270 minutos).
- **Herramientas requeridas:** Flutter SDK 3.22+, Dart SDK 3.4+, Java OpenJDK 21 LTS (o 17 LTS), Spring Boot 3.3+, H2 Database (en memoria), Android Studio (con SDK y AVD), Visual Studio Code con extensiones Flutter y Spring Boot, Git.

  Metas de Aprendizaje:

  1. Diseñar e implementar un servidor RESTful en Spring Boot 3 bajo arquitectura multicapa estándar (`domain`, `controller`, `business`, `data`), capaz de recibir tanto transacciones JSON como cargas útiles de formularios binarios multipart/form-data.

  2. Implementar el patrón de arquitectura móvil **Offline-First** en Flutter, utilizando SQLite local (`sqflite`) como única fuente de verdad inmediata para la interfaz gráfica y administrando colas de sincronización asíncrona hacia el servidor.

  3. Integrar sensores físicos de hardware (antena satelital GPS y cámara óptica), gestionando rigurosamente los permisos nativos en tiempo de ejecución en Android e iOS.

Contexto de Negocio: FieldTech Móvil

La compañía de infraestructura de telecomunicaciones **FieldTech** gestiona miles de kilómetros de redes de fibra óptica y postes en zonas rurales y de montaña. Los técnicos asignados a inspección reportan que en más del 60% de sus rutas no existe señal celular (3G/4G/5G).

El sistema actual en papel provoca pérdidas de evidencias, transcripciones erróneas y desajustes en las ubicaciones cartográficas. FieldTech requiere que usted desarrolle una aplicación móvil que permita:

- Levantar coordenadas GPS precisas con un solo toque.
- Tomar fotografías del desperfecto directamente con la cámara del teléfono.
- Guardar la inspección en la base de datos local SQLite del teléfono instantáneamente.
- Detectar conectividad a Internet y pulsar un botón de sincronización para enviar todas las inspecciones acumuladas al backend Spring Boot 3 de la oficina central.

Estructura General de Carpetas del Proyecto

Usted trabajará en dos proyectos desacoplados organizados de la siguiente manera:

```
laboratorio_15_workspace/
■
■■■ fieldtech-backend/            <-- Proyecto Spring Boot 3 (Maven / Java 21)
■   ■■■ pom.xml
■   ■■■ src/
■       ■■■ main/
■           ■■■ java/com/fieldtech/
■           ■   ■■■ domain/       <-- Entidades JPA y DTOs
■           ■   ■■■ data/         <-- Interfaces de Repositorio Spring Data
■           ■   ■■■ business/     <-- Servicios y Lógica de Negocio
■           ■   ■■■ controller/   <-- Controladores REST y Endpoints
■           ■■■ resources/
■               ■■■ application.properties
■               ■■■ schema.sql
■               ■■■ data.sql
■
■■■ fieldtech_mobile/             <-- Aplicación Móvil en Flutter 3
   ■■■ pubspec.yaml
   ■■■ android/app/src/main/AndroidManifest.xml
   ■■■ ios/Runner/Info.plist
   ■■■ lib/
       ■■■ models/               <-- DTOs y Modelos de Datos en Dart
       ■■■ data/                 <-- DatabaseHelper Singleton con sqflite
       ■■■ services/             <-- Servicios de GPS, Cámara y Red REST
```

```
      ■■■ screens/              <-- Pantallas e Interfaces de Usuario
```

Parte 1: Construcción del Backend en Spring Boot 3 (`fieldtech-backend`)

Siguiendo el estándar de arquitectura multicapa del curso, implementaremos el servidor de recepción de incidencias.

1.1 Configuración de Dependencias (`pom.xml`)

Cree su proyecto Spring Boot configurando el archivo `pom.xml` con Java 21:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.4</version>
        <relativePath/>
    </parent>
    <groupId>com.fieldtech</groupId>
    <artifactId>fieldtech-backend</artifactId>
    <version>1.0.0</version>
    <name>fieldtech-backend</name>
    <description>Servidor RESTful para Recepción de Incidencias Móviles</description>

    <properties>
        <!-- Sustituya por su version local instalada (ej. 17 o 21) -->
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
```

```
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>
</project>
```

1.2 Configuración de Propiedades (`application.properties`)

Configure la base de datos en memoria y la cuota de carga de archivos en `src/main/resources/application.properties`:

```
spring.application.name=fieldtech-backend
server.port=8080

# Base de datos H2 en Memoria
spring.datasource.url=jdbc:h2:mem:fieldtechdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=none
spring.h2.console.enabled=true
spring.sql.init.mode=always

# Configuracion de Tamano para Carga de Evidencias Fotograficas
spring.servlet.multipart.max-file-size=15MB
spring.servlet.multipart.max-request-size=20MB
```

> [!NOTE] Copie los archivos `schema.sql` y `data.sql` ubicados en la carpeta `Evaluaciones/Laboratorios/Laboratorio_15/resources/db/` dentro de `src/main/resources/` de su proyecto Spring Boot.

1.3 Entidad de Dominio (`com.fieldtech.domain.Incidencia`)

```java
package com.fieldtech.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "incidencias")
public class Incidencia {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String titulo;

    @Column(nullable = false, length = 500)
    private String descripcion;

    @Column(nullable = false, length = 50)
    private String categoria;

    @Column(nullable = false)
    private Double latitud;

    @Column(nullable = false)
    private Double longitud;

    @Column(name = "foto_url")
    private String fotoUrl;

    @Column(nullable = false, length = 30)
    private String estado;

    @Column(name = "fecha_reporte")
    private LocalDateTime fechaReporte;

    @Column(name = "tecnico_codigo", nullable = false, length = 50)
    private String tecnicoCodigo;

    public Incidencia() {}

    // Getters y Setters completos
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
```

```
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }

    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; }

    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) { this.categoria = categoria; }

    public Double getLatitud() { return latitud; }
    public void setLatitud(Double latitud) { this.latitud = latitud; }

    public Double getLongitud() { return longitud; }
    public void setLongitud(Double longitud) { this.longitud = longitud; }

    public String getFotoUrl() { return fotoUrl; }
    public void setFotoUrl(String fotoUrl) { this.fotoUrl = fotoUrl; }

    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }

    public LocalDateTime getFechaReporte() { return fechaReporte; }
    public void setFechaReporte(LocalDateTime fechaReporte) {
        this.fechaReporte = fechaReporte;
    }

    public String getTecnicoCodigo() { return tecnicoCodigo; }
    public void setTecnicoCodigo(String tecnicoCodigo) {
        this.tecnicoCodigo = tecnicoCodigo;
    }
}
```

1.4 Capa de Datos / Repositorio (`com.fieldtech.data.IncidenciaRepository`)

```
package com.fieldtech.data;

import com.fieldtech.domain.Incidencia;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface IncidenciaRepository extends JpaRepository<Incidencia, Long> {
    List<Incidencia> findByTecnicoCodigoOrderByFechaReporteDesc(String codigo);
```

```
}
```

1.5 Capa de Negocio / Servicio (`com.fieldtech.business.IncidenciaService`)

```
package com.fieldtech.business;

import com.fieldtech.data.IncidenciaRepository;
import com.fieldtech.domain.Incidencia;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.io.IOException;
import java.time.LocalDateTime;
import java.util.List;
import java.util.UUID;

@Service
public class IncidenciaService {

    private final IncidenciaRepository repository;

    public IncidenciaService(IncidenciaRepository repository) {
        this.repository = repository;
    }

    public List<Incidencia> listarTodas() {
        return repository.findAll();
    }

    public Incidencia registrar(String titulo, String descripcion,
                                String categoria, Double latitud,
                                Double longitud, String tecnicoCodigo,
                                MultipartFile foto) {
        Incidencia inc = new Incidencia();
        inc.setTitulo(titulo);
        inc.setDescripcion(descripcion);
        inc.setCategoria(categoria);
        inc.setLatitud(latitud);
        inc.setLongitud(longitud);
        inc.setTecnicoCodigo(tecnicoCodigo);
        inc.setEstado("REGISTRADA");
        inc.setFechaReporte(LocalDateTime.now());

        if (foto != null && !foto.isEmpty()) {
            // Genera una ruta o identificador simulado de guardado
```

```
            String nombreArchivo = UUID.randomUUID() + "_" + foto.getOriginalFilename();
            inc.setFotoUrl("uploads/evidencias/" + nombreArchivo);
        } else {
            inc.setFotoUrl(null);
        }

        return repository.save(inc);
    }
}
```

1.6 Capa de Controladores REST (`com.fieldtech.controller.IncidenciaController`)

```
package com.fieldtech.controller;

import com.fieldtech.business.IncidenciaService;
import com.fieldtech.domain.Incidencia;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import java.util.List;

@RestController
@RequestMapping("/api/v1/incidencias")
@CrossOrigin(origins = "*") // Habilita consumo desde emuladores y dispositivos
public class IncidenciaController {

    private final IncidenciaService service;

    public IncidenciaController(IncidenciaService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<List<Incidencia>> listar() {
        return ResponseEntity.ok(service.listarTodas());
    }

    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<Incidencia> crearIncidenciaConFoto(
            @RequestParam("titulo") String titulo,
            @RequestParam("descripcion") String descripcion,
            @RequestParam("categoria") String categoria,
```

```
            @RequestParam("latitud") Double latitud,
            @RequestParam("longitud") Double longitud,
            @RequestParam("tecnicoCodigo") String tecnicoCodigo,
            @RequestParam(value = "foto", required = false) MultipartFile foto
    ) {
        Incidencia guardada = service.registrar(
            titulo, descripcion, categoria, latitud, longitud, tecnicoCodigo, foto
        );
        return ResponseEntity.status(HttpStatus.CREATED).body(guardada);
    }
}
```

Parte 2: Construcción de la Aplicación Móvil en Flutter (`fieldtech_mobile`)

2.1 Inicialización del Proyecto y Dependencias

Abra una consola en la raíz de su área de trabajo y ejecute:

```
flutter create --org com.fieldtech fieldtech_mobile
```

Abra el archivo `pubspec.yaml` de la aplicación móvil e incorpore las dependencias requeridas:

```
name: fieldtech_mobile
description: "App Móvil de Gestión de Incidencias Offline-First con GPS y Cámara"
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.4.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.3.3+1
  path: ^1.9.0
  geolocator: ^12.0.0
  image_picker: ^1.1.2
```

```
  http: ^1.2.1
  shared_preferences: ^2.2.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^4.0.0
```

Instale los paquetes ejecutando:

```
flutter pub get
```

2.2 Configuración de Permisos Nativos de Hardware

> [!IMPORTANT] Si omite este paso, el sistema operativo cerrará la aplicación de forma forzada en cuanto intente solicitar las coordenadas satelitales o abrir la cámara.

1. Configuración de Android (`android/app/src/main/AndroidManifest.xml`):
Abra el manifiesto y declare los permisos antes de la etiqueta `<application>`:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Permisos de Ubicacion Satelital (GPS) -->
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

    <!-- Permiso de Camara Fotografica -->
    <uses-permission android:name="android.permission.CAMERA" />

    <application
        android:label="FieldTech Móvil"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher"
        android:usesCleartextTraffic="true">
        ...
    </application>
</manifest>
```

> [!TIP] Observe el atributo `android:usesCleartextTraffic="true"`. Es fundamental para permitir peticiones HTTP sin cifrar hacia su servidor local de desarrollo en

> `http://10.0.2.2:8080`.

2. Configuración de iOS (`ios/Runner/Info.plist`):

Incorpore las justificaciones de uso dentro del bloque `<dict>`:

```
    <key>NSLocationWhenInUseUsageDescription</key>
  <string>FieldTech requiere su ubicacion para georreferenciar la incidencia.</string>
  <key>NSCameraUsageDescription</key>
  <string>FieldTech requiere la camara para capturar evidencia fotografica.</string>
  <key>NSPhotoLibraryUsageDescription</key>
  <string>FieldTech requiere la galeria para seleccionar fotos de archivo.</string>
```

2.3 Persistencia Relacional Local: SQLite Singleton (`lib/data/database_helper.dart`)

Implementaremos el gestor de base de datos local embebido que servirá como fuente de verdad inmediata en el teléfono.

```
// lib/data/database_helper.dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {
 static final DatabaseHelper instance = DatabaseHelper._init();
 static Database? _database;

 DatabaseHelper._init();

 Future<Database> get database async {
   if (_database != null) return _database!;
   _database = await _initDB('fieldtech_offline.db');
   return _database!;
 }

 Future<Database> _initDB(String filePath) async {
   final dbPath = await getDatabasesPath();
   final path = join(dbPath, filePath);

   return await openDatabase(
     path,
     version: 1,
```

```
    onCreate: _createDB,
  );
}

Future<void> _createDB(Database db, int version) async {
  await db.execute('''
    CREATE TABLE incidencias_locales (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      titulo TEXT NOT NULL,
      descripcion TEXT NOT NULL,
      categoria TEXT NOT NULL,
      latitud REAL NOT NULL,
      longitud REAL NOT NULL,
      foto_path TEXT,
      is_synced INTEGER NOT NULL DEFAULT 0,
      fecha_creacion TEXT NOT NULL
    )
  ''');
}

Future<int> insertar(Map<String, dynamic> row) async {
  final db = await instance.database;
  return await db.insert('incidencias_locales', row);
}

Future<List<Map<String, dynamic>>> obtenerTodas() async {
  final db = await instance.database;
  return await db.query('incidencias_locales', orderBy: 'id DESC');
}

Future<List<Map<String, dynamic>>> obtenerPendientes() async {
  final db = await instance.database;
  return await db.query(
    'incidencias_locales',
    where: 'is_synced = ?',
    whereArgs: [0],
  );
}

Future<int> marcarSincronizado(int id) async {
  final db = await instance.database;
  return await db.update(
    'incidencias_locales',
    {'is_synced': 1},
    where: 'id = ?',
    whereArgs: [id],
```

```
    );
  }
}
```

2.4 Servicio de Acceso a Hardware: GPS y Cámara (`lib/services/hardware_service.dart`)

```
// lib/services/hardware_service.dart
import 'dart:io';
import 'package:geolocator/geolocator.dart';
import 'package:image_picker/image_picker.dart';

class HardwareService {
  final ImagePicker _picker = ImagePicker();

  Future<Position> capturarUbicacionGPS() async {
    bool servicioHabilitado = await Geolocator.isLocationServiceEnabled();
    if (!servicioHabilitado) {
      throw Exception('El sensor GPS del teléfono está desactivado.');
    }

    LocationPermission permiso = await Geolocator.checkPermission();
    if (permiso == LocationPermission.denied) {
      permiso = await Geolocator.requestPermission();
      if (permiso == LocationPermission.denied) {
        throw Exception('El usuario denegó el acceso al GPS.');
      }
    }

    if (permiso == LocationPermission.deniedForever) {
      throw Exception('Permisos GPS denegados permanentemente en el sistema.');
    }

    return await Geolocator.getCurrentPosition(
      locationSettings: const LocationSettings(
        accuracy: LocationAccuracy.high,
        timeLimit: Duration(seconds: 12),
      ),
    );
  }

  Future<File?> tomarFotografia() async {
```

```
    final XFile? imagen = await _picker.pickImage(
      source: ImageSource.camera,
      maxWidth: 1600,
      maxHeight: 1200,
      imageQuality: 80, // Compresion optimizada para ahorro de red movil
    );

    if (imagen == null) return null;
    return File(imagen.path);
  }
}
```

2.5 Servicio de Sincronización REST Multipart (`lib/services/sync_service.dart`)

```
// lib/services/sync_service.dart
import 'dart:io';
import 'package:http/http.dart' as http;
import '../data/database_helper.dart';

class SyncService {
  final String backendUrl;
  final DatabaseHelper dbHelper = DatabaseHelper.instance;

  SyncService({required this.backendUrl});

  Future<int> ejecutarSincronizacion(String codigoTecnico) async {
    final List<Map<String, dynamic>> pendientes = await dbHelper.obtenerPendientes();
    if (pendientes.isEmpty) return 0;

    int enviadasConExito = 0;

    for (final inc in pendientes) {
      final int localId = inc['id'] as int;
      final uri = Uri.parse('$backendUrl/api/v1/incidencias');
      final request = http.MultipartRequest('POST', uri);

      request.fields['titulo'] = inc['titulo'];
      request.fields['descripcion'] = inc['descripcion'];
      request.fields['categoria'] = inc['categoria'];
      request.fields['latitud'] = inc['latitud'].toString();
      request.fields['longitud'] = inc['longitud'].toString();
      request.fields['tecnicoCodigo'] = codigoTecnico;
```

```
      final String? rutaFoto = inc['foto_path'];
      if (rutaFoto != null && File(rutaFoto).existsSync()) {
        request.files.add(
          await http.MultipartFile.fromPath('foto', rutaFoto),
        );
      }

      try {
        final streamedRes = await request.send().timeout(
          const Duration(seconds: 15),
        );
        if (streamedRes.statusCode == 200 || streamedRes.statusCode == 201) {
          await dbHelper.marcarSincronizado(localId);
          enviadasConExito++;
        }
      } catch (e) {
        // En caso de corte de señal, interrumpimos la cola para reintentar luego
        break;
      }
    }
    return enviadasConExito;
  }
}
```

2.6 Interfaz Gráfica: Pantalla Principal y Formulario en Vivo (`lib/main.dart`)

```
// lib/main.dart
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';
import 'data/database_helper.dart';
import 'services/hardware_service.dart';
import 'services/sync_service.dart';

void main() {
 WidgetsFlutterBinding.ensureInitialized();
 runApp(const FieldTechApp());
}

class FieldTechApp extends StatelessWidget {
 const FieldTechApp({super.key});
```

```
 @override
 Widget build(BuildContext context) {
   return MaterialApp(
     title: 'FieldTech Móvil',
     debugShowCheckedModeBanner: false,
     theme: ThemeData(
       useMaterial3: true,
       colorScheme: ColorScheme.fromSeed(
         seedColor: const Color(0xFF0284C7),
         brightness: Brightness.light,
       ),
     ),
     home: const IncidenciasScreen(),
   );
 }
}

class IncidenciasScreen extends StatefulWidget {
 const IncidenciasScreen({super.key});

 @override
 State<IncidenciasScreen> createState() => _IncidenciasScreenState();
}

class _IncidenciasScreenState extends State<IncidenciasScreen> {
 final HardwareService _hardware = HardwareService();
 // 10.0.2.2 apunta a la maquina anfitriona desde el emulador de Android
 final SyncService _syncService = SyncService(
   backendUrl: 'http://10.0.2.2:8080',
 );

 List<Map<String, dynamic>> _incidencias = [];
 bool _cargando = false;

 @override
 void initState() {
   super.initState();
   _cargarIncidenciasLocales();
 }

 Future<void> _cargarIncidenciasLocales() async {
   final datos = await DatabaseHelper.instance.obtenerTodas();
   setState(() => _incidencias = datos);
 }
```

```
Future<void> _sincronizar() async {
  setState(() => _cargando = true);
  final total = await _syncService.ejecutarSincronizacion('TEC-CR-001');
  await _cargarIncidenciasLocales();
  setState(() => _cargando = false);

  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          total > 0
              ? '¡Sincronización exitosa! ($total registros enviados).'
              : 'No hay registros pendientes o el servidor no respondió.',
        ),
        backgroundColor: total > 0 ? Colors.green[700] : Colors.orange[800],
      ),
    );
  }
}

void _abrirFormularioModal() {
  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    builder: (ctx) => NuevaIncidenciaModal(
      onGuardado: _cargarIncidenciasLocales,
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('FieldTech Inspecciones'),
      actions: [
        IconButton(
          icon: _cargando
              ? const SizedBox(
                  width: 20,
                  height: 20,
                  child: CircularProgressIndicator(
                    color: Colors.white,
                    strokeWidth: 2,
                  ),
                )
```

```
              : const Icon(Icons.sync),
          tooltip: 'Sincronizar con Servidor',
          onPressed: _cargando ? null : _sincronizar,
        ),
      ],
    ),
    body: _incidencias.isEmpty
        ? const Center(
            child: Text(
              'No hay inspecciones registradas.\nPresione + para crear una.',
              textAlign: TextAlign.center,
              style: TextStyle(color: Colors.grey, fontSize: 16),
            ),
          )
        : ListView.builder(
            itemCount: _incidencias.length,
            itemBuilder: (ctx, index) {
              final item = _incidencias[index];
              final bool sincronizado = item['is_synced'] == 1;
              return Card(
                margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
                elevation: 2,
                child: ListTile(
                  leading: CircleAvatar(
                    backgroundColor: sincronizado ? Colors.green[100] : Colors.amber[100],
                    child: Icon(
                      sincronizado ? Icons.cloud_done : Icons.cloud_off,
                      color: sincronizado ? Colors.green[800] : Colors.amber[900],
                    ),
                  ),
                  title: Text(
                    item['titulo'],
                    style: const TextStyle(fontWeight: FontWeight.bold),
                  ),
                  subtitle: Text(
                    '${item['categoria']} | Lat: ${item['latitud']}, Lon: ${item['longitud']}',
                  ),
                  trailing: Text(
                    sincronizado ? 'En Servidor' : 'Offline',
                    style: TextStyle(
                      fontSize: 12,
                      color: sincronizado ? Colors.green[700] : Colors.amber[900],
                      fontWeight: FontWeight.w600,
                    ),
                  ),
                ),
```

```
                );
              },
            ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _abrirFormularioModal,
        icon: const Icon(Icons.add_a_photo),
        label: const Text('Nueva Incidencia'),
      ),
    );
  }
}

class NuevaIncidenciaModal extends StatefulWidget {
  final VoidCallback onGuardado;
  const NuevaIncidenciaModal({super.key, required this.onGuardado});

  @override
  State<NuevaIncidenciaModal> createState() => _NuevaIncidenciaModalState();
}

class _NuevaIncidenciaModalState extends State<NuevaIncidenciaModal> {
  final _tituloController = TextEditingController();
  final _descController = TextEditingController();
  final HardwareService _hardware = HardwareService();

  String _categoria = 'INFRAESTRUCTURA';
  Position? _posicion;
  File? _foto;
  bool _obteniendoGps = false;

  Future<void> _capturarGPS() async {
    setState(() => _obteniendoGps = true);
    try {
      final pos = await _hardware.capturarUbicacionGPS();
      setState(() => _posicion = pos);
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(e.toString()), backgroundColor: Colors.red),
        );
      }
    } finally {
      setState(() => _obteniendoGps = false);
    }
  }
```

```
Future<void> _capturarCamara() async {
  final imagen = await _hardware.tomarFotografia();
  if (imagen != null) {
    setState(() => _foto = imagen);
  }
}

Future<void> _guardarEnSQLite() async {
  if (_tituloController.text.trim().isEmpty || _posicion == null) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Debe ingresar un título y capturar las coordenadas GPS.'),
        backgroundColor: Colors.red,
      ),
    );
    return;
  }

  final row = {
    'titulo': _tituloController.text.trim(),
    'descripcion': _descController.text.trim(),
    'categoria': _categoria,
    'latitud': _posicion!.latitude,
    'longitud': _posicion!.longitude,
    'foto_path': _foto?.path,
    'is_synced': 0,
    'fecha_creacion': DateTime.now().toIso8601String(),
  };

  await DatabaseHelper.instance.insertar(row);
  widget.onGuardado();
  if (mounted) Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  return Padding(
    padding: EdgeInsets.only(
      bottom: MediaQuery.of(context).viewInsets.bottom + 20,
      left: 20,
      right: 20,
      top: 20,
    ),
    child: SingleChildScrollView(
      child: Column(
        mainAxisSize: MainAxisSize.min,
```

```
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Text(
            'Registrar Incidencia en Campo',
            style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 12),
          TextField(
            controller: _tituloController,
            decoration: const InputDecoration(
              labelText: 'Título de la Incidencia',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 10),
          TextField(
            controller: _descController,
            maxLines: 2,
            decoration: const InputDecoration(
              labelText: 'Descripción técnica',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),
          DropdownButtonFormField<String>(
            value: _categoria,
            decoration: const InputDecoration(border: OutlineInputBorder()),
            items: const [
              DropdownMenuItem(value: 'INFRAESTRUCTURA', child: Text('Infraestructura')),
              DropdownMenuItem(value: 'RED_ACCESO', child: Text('Red de Acceso')),
              DropdownMenuItem(value: 'TORRE_RADIO', child: Text('Torre de Radio')),
            ],
            onChanged: (v) => setState(() => _categoria = v!),
          ),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(
                child: ElevatedButton.icon(
                  onPressed: _obteniendoGps ? null : _capturarGPS,
                  icon: _obteniendoGps
                      ? const SizedBox(
                          width: 16,
                          height: 16,
                          child: CircularProgressIndicator(strokeWidth: 2),
                        )
```

```
                        : const Icon(Icons.my_location),
                    label: Text(
                      _posicion != null
                          ? 'GPS: ${_posicion!.latitude.toStringAsFixed(3)}, ${_posicion!.longitude.
                          : 'Obtener GPS',
                    ),
                  ),
                ),
                const SizedBox(width: 10),
                IconButton.filledTonal(
                  onPressed: _capturarCamara,
                  icon: const Icon(Icons.camera_alt),
                  tooltip: 'Tomar Fotografía',
                ),
              ],
            ),
            if (_foto != null) ...[
              const SizedBox(height: 10),
              ClipRRect(
                borderRadius: BorderRadius.circular(8),
                child: Image.file(_foto!, height: 120, fit: BoxFit.cover),
              ),
            ],
            const SizedBox(height: 16),
            FilledButton(
              onPressed: _guardarEnSQLite,
              child: const Text('Guardar Localmente (Offline)'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Parte 3: Verificación y Depuración de Errores Comunes de Desarrollo Móvil

3.1 Loopback de Red en Android (`10.0.2.2` vs `localhost`)

- **Problema:** La aplicación móvil no logra contactar al backend y arroja `SocketException: Connection refused`.
- **Causa:** En emuladores de Android, `localhost` o `127.0.0.1` apunta al propio sistema Android virtualizado.
- **Solución:** Configure siempre `http://10.0.2.2:8080`. Si ejecuta la aplicación en un teléfono real conectado por USB, configure la IP privada de su máquina anfitriona (ej. `http://192.168.1.45:8080`) y asegúrese de que ambos equipos compartan la misma red Wi-Fi.

3.2 Simulación de Coordenadas Satelitales en AVD

- **Problema:** Al pulsar *Obtener GPS*, la llamada se agota por timeout (`TimeoutException`).
- **Causa:** El emulador de Android Studio no cuenta con una antena de satélite real.
- **Solución:** Abra los controles extendidos del emulador pulsando el botón `...` en la barra lateral del AVD, navegue a la pestaña **Location**, escriba manualmente coordenadas (ej. Latitud: `9.9322`, Longitud: `-84.0795`) y pulse el botón **Set Location**.

Parte 4: Reto Autónomo de Refuerzo

Para consolidar sus competencias en persistencia y diseño móvil, implemente el siguiente requerimiento sin alterar la arquitectura base:

- [ ] **Filtro Segmentado de Incidencias:** Incorpore en la barra superior o bajo el AppBar un control visual `SegmentedButton` en Flutter con tres opciones: `Todas`, `Offline (Pendientes)` y `Sincronizadas`. La lista reactiva debe refrescarse instantáneamente consultando a SQLite mediante cláusulas `WHERE` condicionales.

Rúbrica de Evaluación Sumativa (100 Puntos)

Criterio	Descripción	Puntos
Arquitectura Multicapa en Spring Boot	Separación rigurosa de paquetes (`domain`, `controller`, `business`, `data`), entidades JPA e inicialización automática con H2 Database.	25 pts
Persistencia Local SQLite en Flutter	Implementación del patrón Singleton con `sqflite`, creación del esquema DDL y almacenamiento inmediato sin conexión.	25 pts
Integración de Sensores de Hardware	Solicitud y validación de permisos en tiempo de ejecución, obtención de coordenadas GPS y captura fotográfica.	25 pts
Sincronización Asíncrona Multipart	Envío en segundo plano de payloads compuestos (datos de texto + binario de foto) hacia el endpoint REST y actualización de `is_synced`.	15 pts
Reto Autónomo y Buenas Prácticas	Implementación del filtro segmentado en UI y cumplimiento de las directrices de código limpio.	10 pts